

Full-Stack Implementation Sequence

Step-by-step modular development order • Architecture & Build Plan

Phase 1 — Authentication & Core Setup

ITEMS 1–11

1	server/config/db.js	MongoDB database connection configuration
2	server/models/User.js	User schema, password hashing logic
3	server/controllers/authController.js	Signup & login handling logic
4	server/routes/authRoutes.js	Authentication API endpoints
5	server/middleware/auth.js	JWT token verification middleware
6	server/server.js	Wire items 1–5 together, verify auth endpoints
7	client/src/api/authApi.js	Frontend HTTP client for auth endpoints
8	client/src/pages/Signup.jsx	User registration interface
9	client/src/pages/Login.jsx	User authentication interface
10	client/src/context/AuthContext.jsx	Global user authentication state management
11	client/src/components/ProtectedRoute.jsx	Route wrapper enforcing authentication



Checkpoint: User can sign up, log in, and auto-redirect if unauthenticated.

Phase 2 — Workspace Management

ITEMS 12–16

12	server/models/Workspace.js	Workspace schema & member mappings
13	server/controllers/workspaceController.js	Workspace CRUD logic
14	server/routes/workspaceRoutes.js	Workspace management API endpoints
15	client/src/api/workspaceApi.js	Frontend API helper for workspace operations
16	client/src/pages/WorkspaceSelect.jsx	Workspace creation & selection interface



Checkpoint: Logged-in user can successfully create or select a workspace.

Phase 3 — Channels & Permissions

ITEMS 17–22

17	server/models/Channel.js	Channel schema associated with workspace
18	server/middleware/roleCheck.js	Role & permissions access control middleware
19	server/controllers/channelController.js	Channel creation & listing controller
20	server/routes/channelRoutes.js	Channel API endpoints
21	client/src/api/channelApi.js	Frontend API service for channels

Phase 4 — Real-Time Chat System		CORE DELIVERABLE • ITEMS 23–33
23	server/models/Message.js	Chat message schema with timestamps & references
24	server/sockets/index.js	Socket.io server initialization & connection handler
25	server/sockets/chatHandlers.js	Real-time message broadcast & listening events
26	server/sockets/presenceHandlers.js	User presence and typing indicator events
27	server/controllers/messageController.js	REST controller for fetching historic chat logs
28	server/routes/messageRoutes.js	HTTP routes for historic chat retrieval
29	client/src/hooks/useSocket.js	Custom React hook for socket lifecycle management
30	client/src/api/messageApi.js	Frontend client for historic message API
31	client/src/components/MessageBubble.jsx	Component for rendering chat messages
32	client/src/components/TypingIndicator.jsx	Component displaying active user typing state
33	client/src/pages/ChatView.jsx	Main UI assembling real-time messaging interface
 Checkpoint: Real-time chat fully functional. Core milestone established.		

Phase 5 — AI Integration (RAG + Function Calling)		ITEMS 34–41
34	server/models/PinnedDoc.js	Schema for pinned contextual knowledge documents
35	server/services/embeddingService.js	Service generating vector embeddings for text
36	server/services/ragService.js	Vector lookup & Retrieval-Augmented Generation service
37	server/controllers/aiController.js	AI query processing & function execution controller
38	server/routes/aiRoutes.js	AI query and pinning endpoint routes
39	client/src/api/aiApi.js	Frontend client service for AI endpoints
40	client/src/components/PinButton.jsx	UI control to pin messages/docs for AI context
41	client/src/components/AIQAPanel.jsx	Interactive AI query panel interface
 Checkpoint: Message pinning and grounded AI Q&A system functioning.		

Phase 6 — Task Management		ITEMS 42–46
42	server/models/Task.js	Task schema with status, assignees, and references
43	server/controllers/taskController.js	Task CRUD operations & AI creation handler
44	server/routes/taskRoutes.js	Task API routes
45	client/src/api/taskApi.js	Frontend service for managing tasks
46	client/src/pages/TaskBoard.jsx	Kanban/List view page for task tracking